



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 编译内核利用已有内核构建 OS

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/3/16-2026/3/18

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
五、实验总结与心得体会.....	14
六、代码清单.....	14

Section 1 实验概述

学习到如何从 16 位的实模式跳转到 32 位的保护模式，然后在平坦模式下运行 32 位程序。同时学习到如何使用 I/O 端口和硬件交互，为后面保护模式编程打下基础。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++ 程序运行、计算机组成原理
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）
 - 代码编译工具：GCC、qemu-system-i386、GDB、nasm（汇编编译器）、dd（文件写入工具）
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：学习从 16 位实模式到 32 位保护模式的基础知识
- 实验任务 2：复现"加载 bootloader"
- 实验任务 3：使用 CHS 模式来读取硬盘
- 实验任务 4：复现“进入保护模式”一节，使用 gdb 或其他 debug 工具调试
- 实验任务 5：编写并执行一个自己定义的 32 位汇编程序

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：了解 LBA 方式读写硬盘、保护模式等基础知识
- 实验预备知识：

▲LBA 模式读写硬盘

- ①使用 in,out 指令对端口进行赋值

```

; in指令
in al, 0x21 ; 表示从0x21端口读取一字节数据到al
in ax, 0x21 ; 表示从端口地址0x21读取1字节数据到al, 从端口地址0x22读取1字节到ah

mov dx, 0x379
in al, dx ; 从端口0x379读取1字节到al

; out指令
out 0x21, al ; 将al的值写入0x21端口
out 0x21, ax ; 将ax的值写入端口地址0x21开始的连续两个字节
mov dx, 0x378
out dx, ax ; 将ah和al分别写入端口0x379和0x378

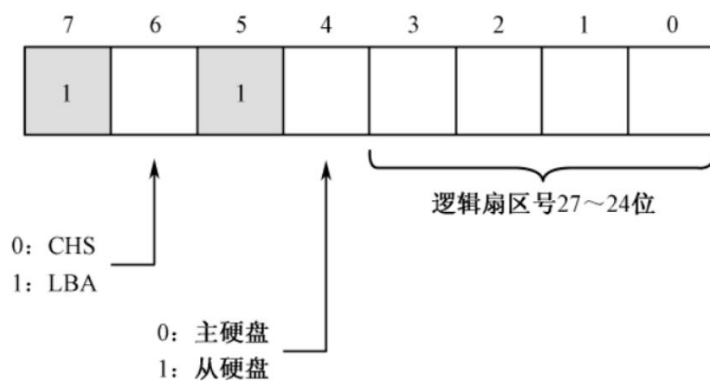
```

②了解 CHS 定位磁盘数据以及 CHS 模式（BIOS 中断，16 位程序，再实模式使用）

③使用 LBA（Logical Block Addressing）模式访问磁盘

！我们的 bootloader 放在磁盘的第 1~5 块上(第 0 块是 MBR),为了加载 MBR,我们就要读取磁盘的第 1~5 块

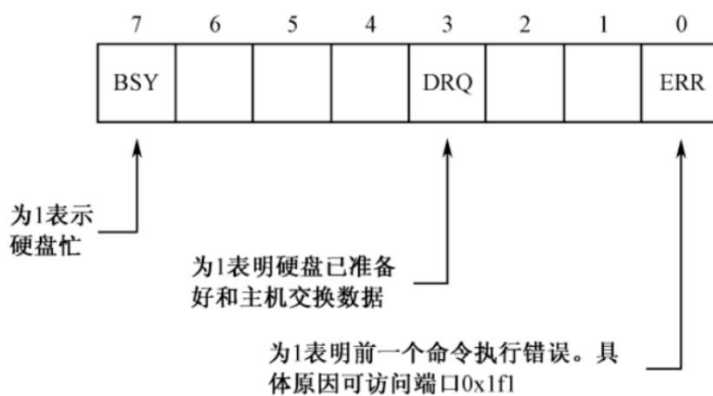
Step1: 设置起始的逻辑扇区号——LBA28, 逻辑扇区的 0~7 位被写入 0x1F3 端口, 8~15 位被写入 0x1F4 端口, 16~23 位被写入 0x1F5 端口, 最后 4 位被写入 0x1F6 端口的低 4 位。0x1F6 的 8 个位的含义有点特殊, 如下所示。



Step2: 将要读取的扇区数量写入 0x1F2 端口（8 位）

Step3: 向 0x1F7 端口写入 0x20, 请求硬盘读

Step4: 等待其他读写操作完成, 硬盘的状态可以从 0x1F7 读入, 读到的 8 个状态位如下所示。



Step5: 从 0x1F0 中连续读入一个扇区的数据（0x1F0 是硬盘接口的数据端口，16 位）

④理解汇编代码

▲保护模式

①概述：，程序运行在自己的段中，保护模式提供了段间的保护机制、访问的内存空间变大、提供了虚拟内存的管理机制

②从实模式到保护模式：

a.保护模式保护的是段地址空间，阻止程序越界访问，因此也被称为段保护

b. 物理地址=f(线性地址)

c.全局描述符表（集中放置段描述符）：Global Descriptor Table，简称 GDT

d.寄存器从 16 位到 32 位

e.寻址：线性地址 = 段地址 + 偏移地址

③进入保护模式：

a. 我们需要在实模式下的 MBR 中编写 16 位进入保护模式的代码，然后再跳转到保护模式，执行接下来的 32 位代码。

b.过程

1. 准备GDT，用lgdt指令加载GDTR信息。

2. 打开第21根地址线。

3. 开启cr0的保护模式标志位。

4. 远跳转，进入保护模式。

----- 实验任务 2（思考题 9） -----

- 任务要求：复现"加载 bootloader"一节
- 思路分析：需要执行远跳转到 0x7e00, 0x0000:0x7e00 表示段地址为 0x0000, 偏移地址为 0x7e00
- 实验步骤：

①编写 bootloader.asm，在给出代码的基础上加入 hello world 部分

```
1  org 0x7e00
2  [bits 16]
3
4  xor ax, ax
5  mov ds, ax
6  mov ss, ax
7  mov es, ax
8  mov fs, ax
9  mov gs, ax
10 mov sp, 0x7e00
11
12 ; 输出 Hello World
13 mov ax, 0xb800
14 mov gs, ax
15
16 mov ah, 0x01
17 mov al, 'H'
18 mov [gs:2 * 0], ax
19
20 mov al, 'e'
21 mov [gs:2 * 1], ax
22
23 mov al, 'l'
24 mov [gs:2 * 2], ax
25
26 mov al, 'l'
27 mov [gs:2 * 3], ax
28
29 mov al, 'o'
30 mov [gs:2 * 4], ax
31
32 mov al, ' '
33 mov [gs:2 * 5], ax
34
35 mov al, 'W'
36 mov [gs:2 * 6], ax
37
38 mov al, 'o'
39 mov [gs:2 * 7], ax
40
41 mov al, 'r'
42 mov [gs:2 * 8], ax
43
44 mov al, 'l'
45 mov [gs:2 * 9], ax
46
47 mov al, 'd'
48 mov [gs:2 * 10], ax
49
50 mov ah, 0x03 ; 青色
51 mov ecx, bootloader_tag_end - bootloader_tag ; 计算字符串长度
52 xor ebx, ebx
53 mov esi, bootloader_tag
54 output_bootloader_tag:
55     mov al, [esi]
56     mov word[gs:bx + 2*12], ax ; 从第12个位置开始输出
57     inc esi
58     add ebx, 2
59     loop output_bootloader_tag
60
61 jmp $ ; 死循环
62
63 bootloader_tag db 'run bootloader'
64 bootloader_tag_end:
```

②编写 mbr.asm，并放入使用 LBA 模式读取硬盘的代码，然后在 MBR 中加载 bootloader 到地址 0x7e00——详见附件

③将 bootloader.asm 和 mbr.asm 编译并且写入扇区，最后使用 qemu 运行

```
lalala@lalala-VirtualBox:~/src/1$ nasm -f bin bootloader.asm -o bootloader.bin
lalala@lalala-VirtualBox:~/src/1$ dd if=bootloader.bin of=hd.img bs=512 count=5
seek=1 conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
136 bytes copied, 0.000125669 s, 1.1 MB/s
lalala@lalala-VirtualBox:~/src/1$ nasm -f bin mbr.asm -o mbr.bin
lalala@lalala-VirtualBox:~/src/1$ dd if=mbr.bin of=hd.img bs=512 count=1 seek=0
conv=notrunc
记录了0+1 的读入
记录了0+1 的写出
136 bytes copied, 0.000150226 s, 905 kB/s
```

● 实验结果展示:

```
lalala@lalala-VirtualBox:~/src/1$ qemu-system-x86_64 -hda hd.img -boot c
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
qemu-system-x86_64: warning: TCG doesn't support requested feature: CPUID.01H:ECX.vmx [bit 5]
[ ]
Hello Worldrun bootloaderbuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
```

----- 实验任务 3（思考题 10） -----

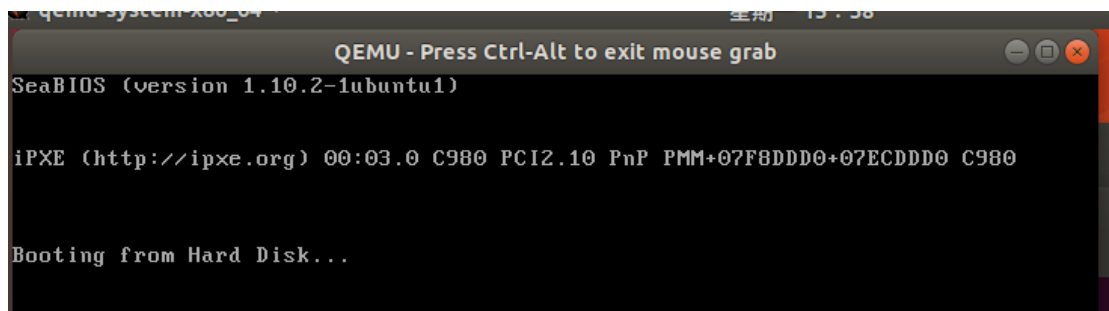
- 任务要求: BIOS 提供了实模式下读取硬盘的中断，其不需要关心具体的 I/O 端口，只需要给出逻辑扇区号对应的磁头（Heads）、扇区（Sectors）和柱面（Cylinder）即可，又被称为 CHS 模式，现在需要使用 CHS 模式来读取硬盘
- 思路分析:

1.注意语法字符的规范

```
MOV CH, al
mov cl, (al >> 2) | cl
```

```
lalala@lalala-VirtualBox:~/src/1_1$ nasm -f bin mbr.asm -o mbr.bin
mbr.asm:33: error: shift operator may only be applied to scalar values
mbr.asm:33: error: '|' operator may only be applied to scalar values
```

2. BIOS 的 int 0x13, ah=0x02 明确要求: 读取的数据存放在 ES:BX 指向的内存, 所以在实验过程中不能破坏 BX 存储的内存偏移地址, 否则会导致 BIOS 无法正确识别想要读取的磁盘地址, 出现下图结果



● 实验步骤:

①学习 LBA 转 CHS 公式以及 int 0x13BIOS 中断

$$\#c = \#lba / (S * H)$$

$$\#h = (\#lba / S) \% H$$

$$\#s = (\#lba \% S) + 1$$

②修改 mbr.asm 代码

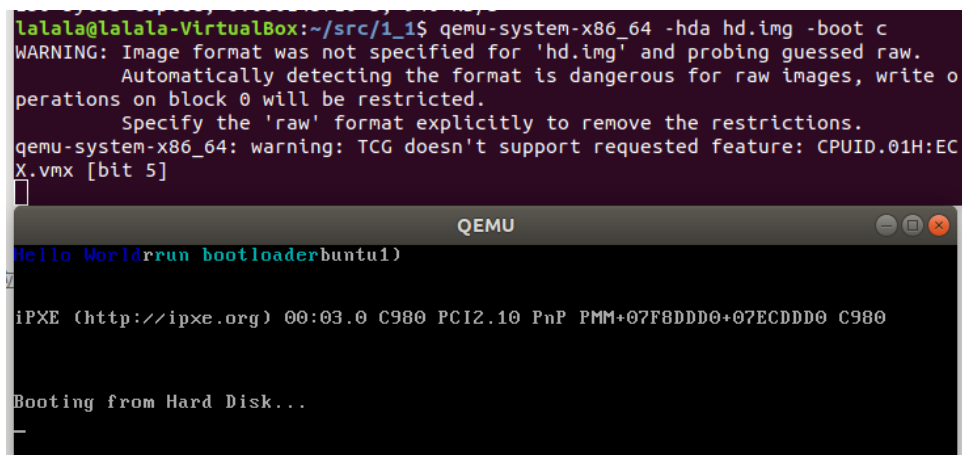
```

1  org 0x7c00
2  [bits 16]
3
4  xor ax, ax
5  mov ds, ax
6  mov es, ax
7  mov ss, ax
8  mov sp, 0x7c00
9
10 mov bx, 0x7e00
11 mov si, 1
12 mov di, 5
13
14 load_bootloader:
15     push di        ; 保存循环计数器
16     push bx        ; ***注意: 保存当前内存偏移地址, 防止计算时破坏***
17
18     ; LBA 转 CHS
19     mov ax, si
20     mov cl, 63     ; 每磁道扇区数
21     div cl
22
23     mov cl, ah
24     inc cl         ; CL = 扇区号
25
26     xor ah, ah
27     mov dl, 16     ; 每柱面磁头数
28     div dl         ; al = 柱面 (CH), ah = 磁头 (DH)
29
30     mov ch, al
31     mov dh, ah     ; DH = 磁头号
32     mov dl, 0x80   ; DL = 驱动器号 (0x80 为硬盘)
33
34     pop bx         ; 恢复目标内存偏移地址到 bx
35     mov ax, 0x0201 ; ah=02 (读), al=01 (读1个扇区)
36     int 0x13
37     jc error
38
39     add bx, 512
40     inc si
41     pop di         ; 恢复循环计数
42     dec di
43     jnz load_bootloader
44
45     jmp 0x0000:0x7e00 ; 跳转执行
46
47 error:
48     jmp $
49
50 times 510 - ($ - $$) db 0
51 db 0x55, 0xaa

```

③重复前面编译、写入扇区以及使用 qemu 运行操作

- 实验结果展示：通过执行前述代码，可得下图结果。



```
lalala@lalala-VirtualBox:~/src/1_1$ qemu-system-x86_64 -hda hd.img -boot c
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
        Automatically detecting the format is dangerous for raw images, write o
perations on block 0 will be restricted.
        Specify the 'raw' format explicitly to remove the restrictions.
qemu-system-x86_64: warning: TCG doesn't support requested feature: CPUID.01H:EC
X.vmx [bit 5]

Hello Worldrun bootloaderbuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
```

-

----- 实验任务 4（思考题 11） -----

- 任务要求：复现“进入保护模式”一节，使用 gdb 或其他 debug 工具在进入保护模式的 4 个重要步骤上设置断点，并结合代码、寄存器的内容等来分析这 4 个步骤

- 思路分析：

1. mbr.asm 中默认 bootloader 大小为 5 个扇区共 2560 字节，但 bootloader.asm 编译得到文件大小不足 2560 字节，会导致 mbr.asm 代码读取硬盘时出现错误陷入死循环，需要手动填充 bootloader.asm 代码至 2560 字节

2. 在编写 mbr.asm 过程中**注意需要保存寄存器**，避免破坏原来的值

- 实验步骤：

①在 boot.inc 里面对内存地址以及段选择子进行规划

```

; ----- Loader (bootloader) 相关常量 -----
; bootloader 占用扇区数
_LOADER_SECTOR_COUNT equ 5
; bootloader 起始扇区号 (MBR 占第0扇区, bootloader 从第1扇区开始)
_LOADER_START_SECTOR equ 1
; bootloader 加载到内存的起始地址
_LOADER_START_ADDRESS equ 0x7e00

; ----- GDT 相关常量 -----
; GDT 起始地址 (紧跟 bootloader 末尾, 0x7e00 + 5*512 = 0x8800)
_GDT_START_ADDRESS equ 0x8800

; ----- 段选择子常量 -----
; 数据段选择子 (GDT 中第1个非空描述符, 索引1, 对应偏移0x08)
_DATA_SELECTOR equ 0x8
; 栈段选择子 (索引2, 偏移0x10)
_STACK_SELECTOR equ 0x10
; 显存段选择子 (索引3, 偏移0x18)
_VIDEO_SELECTOR equ 0x18
; 代码段选择子 (索引4, 偏移0x20)
_CODE_SELECTOR equ 0x20

```

②编写 mbr.asm 代码（详见附件）

▲MBR 中循环读取 bootloader 的核心逻辑，其中 **push ax** 和 **push bx** 时给硬盘读取函数传参，16 位实模式下没有寄存器传参的标准，这里用**栈传参**

```

1  ; 循环读取 5 个扇区的 bootloader
2  load_bootloader:
3      push ax
4      push bx
5      call asm_read_hard_disk ; 调用硬盘读取函数
6      add sp, 4 ; 清除栈中2个参数
7      inc ax
8      add bx, 512
9      loop load_bootloader
10
11 ; 远跳转到 bootloader 执行
12 jmp 0x0000:0x7e00

```

③编写 bootloader.asm 代码（详见附件）

▲在 GDT 中依次放入 0 描述符，数据段描述符、堆栈段描述符、显存段描述符和代码段描述符

```

1 ; ----- 初始化 GDT (全局描述符表) -----
2 ; GDT 起始地址: 0x8800 (紧跟 bootloader 末尾)
3 ; 1. 空描述符 (GDT 第0个描述符必须全0)
4 mov dword [GDT_START_ADDRESS+0x00], 0x00
5 mov dword [GDT_START_ADDRESS+0x04], 0x00
6
7 ; 2. 平坦模式数据段描述符 (0~4GB)
8 mov dword [GDT_START_ADDRESS+0x08], 0x0000ffff ; 基地址0, 段界限0xFFFF
9 mov dword [GDT_START_ADDRESS+0x0c], 0x00cf9200 ; G=1 (4KB粒度), P=1 (段存在), DPL=0, TYPE=0x2 (可读写)
10
11 ; 3. 栈段描述符 (0~4GB)
12 mov dword [GDT_START_ADDRESS+0x10], 0x00000000 ; 基地址0, 段界限0
13 mov dword [GDT_START_ADDRESS+0x14], 0x00409600 ; G=0 (1字节粒度), P=1, DPL=0, TYPE=0x6 (栈段)
14
15 ; 4. 显存段描述符 (0x8000~0xBFFFF)
16 mov dword [GDT_START_ADDRESS+0x18], 0x8000ffff ; 基地址0x0008000, 段界限0x07FFF
17 mov dword [GDT_START_ADDRESS+0x1c], 0x0040920b ; G=0, P=1, DPL=0, TYPE=0x2
18
19 ; 5. 平坦模式代码段描述符 (0~4GB)
20 mov dword [GDT_START_ADDRESS+0x20], 0x0000ffff ; 基地址0, 段界限0xFFFF
21 mov dword [GDT_START_ADDRESS+0x24], 0x00cf9800 ; G=1, P=1, DPL=0, TYPE=0x8 (只执行)
22

```

▲加载 GDTR

```

1 ; ----- 加载 GDTR 寄存器 -----
2 ; 使用一个48位的变量来暂时存放GDTR的内容
3 pgdt dw 0
4      dd GDT_START_ADDRESS
5
6 ; 初始化 GDTR
7 mov word [pgdt], 39 ;描述表地界限
8 lgdt [pgdt]

```

▲打开 A20 的地址线

```

1 ; -----打开 A20 地址线 (访问更多的内存) -----
2 in al, 0x92 ; 南桥芯片内的端口
3 or al, 0000_0010B
4 out 0x92, al ; 打开A20

```

▲开启保护模式并且通过远跳转进入保护模式

```

1  ; ----- 开启保护模式-----
2  ; PE (Protection Enable) 位
3  ; - PE 是 cr0 寄存器的第0位
4  ; - PE=0: CPU 工作在实模式
5  ; - PE=1: CPU 工作在保护模式
6  cli                                ; 中断机制尚未工作
7  mov eax, cr0
8  or  eax, 1
9  mov cr0, eax                      ; 设置PE位
10
11 ; ----- 远跳转进入保护模式-----
12 jmp dword CODE_SELECTOR:protect_mode_begin

```

▲进入保护模式后将选择子放入对应的段寄存器，输出“enter protect mode”

④调试环境准备：

a.强制 16 位模式

b.在 Bootloader 入口设置断点

c.开启自动反汇编查看代码

```

(gdb) set architecture i8086
The target architecture is assumed to be i8086
(gdb) b *0x7e00
Breakpoint 1 at 0x7e00
(gdb) c
Continuing.

Breakpoint 1, 0x00007e00 in ?? ()
(gdb) display/5i $pc
1: x/5i $pc
=> 0x7e00:    mov     $0xe88eb800,%eax
    0x7e05:    mov     $0x3,%ah
    0x7e07:    mov     $0xe,%cx
    0x7e0b:    add     %al,(%eax)
    0x7e0d:    xor     %bx,%bx

```

⑤关键步骤 1：加载 GDT

▲看见 loop 0x7e16 时，可以输入 until *0x7e25 跳出循环

```

=> 0x7e1d:    inc     %si
    0x7e1f:    add     $0x2,%bx
    0x7e23:    loop    0x7e16
    0x7e25:    movw    $0x8800,(%esi)
    0x7e2a:    add     %al,(%eax)

```

▲执行 lgdtw 指令之前，确认内存里面有数据（GDT 全局描述表）

```

1: x/5i $pc
=> 0x7e8b:      lgdtl   (%esi)
    0x7e8e:      jg      0x7f0e
    0x7e90:      in      $0x92,%al
    0x7e92:      or      $0x2,%al
    0x7e94:      out     %al,$0x92
(gdb) x/40bx 0x8800
0x8800: 0x00  0x00  0x00  0x00  0x00  0x00  0x00  0x00
0x8808: 0xff  0xff  0x00  0x00  0x00  0x92  0xcf  0x00
0x8810: 0x00  0x00  0x00  0x00  0x00  0x96  0x40  0x00
0x8818: 0xff  0x7f  0x00  0x80  0x0b  0x92  0x40  0x00
0x8820: 0xff  0xff  0x00  0x00  0x00  0x98  0xcf  0x00
(gdb)

```

▲执行 lgdtw 命令后 GDTR 基址变为 00008800，界限（limit）为 0027

```

TR=0000 00000000 00001111 0
GDT= 00008800 00000027

```

⑥关键步骤 2：打开 A20 地址线

在执行 or \$0x2,%al 指令后查看 al 值为 0x02，标志第 21 根地址线解锁

```

$1 = 0x0
(gdb) p/x $al
$2 = 0x2

```

⑦关键步骤 3：改变 PE 位，切换模式

▲设置断点，运行后查看 \$cr0

```

Breakpoint 2 at 0x7e97
(gdb) b *0x7e97
Breakpoint 3 at 0x7e97
(gdb) c
Continuing.

Breakpoint 3, 0x00007e97 in ?? ()
1: x/5i $pc
=> 0x7e97:      mov     %cr0,%eax
    0x7e9a:      or      $0x1,%ax
    0x7e9e:      mov     %eax,%cr0
    0x7ea1:      ljmpw   $0x0,$0x7ea9
    0x7ea7:      and     %al,(%eax)
(gdb) si
0x00007e9a in ?? ()
1: x/5i $pc
=> 0x7e9a:      or      $0x1,%ax
    0x7e9e:      mov     %eax,%cr0
    0x7ea1:      ljmpw   $0x0,$0x7ea9
    0x7ea7:      and     %al,(%eax)
    0x7ea9:      mov     $0x8,%eax
(gdb) monitor info registers

```

▲注意这一步需要使用 **monitor info registers** 直接向 QEMU 内核提问，如果使用 p/x \$cr0 可能由于延迟问题造成 cr0 值不改变

```

IDT= 00000000 000003ff
CR0=00000011 CR2=00000000 CR3=00000000 CR4=00000000
DR0=00000000 DR1=00000000 DR2=00000000 DR3=00000000
DR6=ffff0ff0 DR7=00000400

```

⑧关键步骤 4：远跳转

a. 执行 `ljmp` 指令

b. 切换到 32 位

c. 查看 `cs` 寄存器是否变为 `0x0020`

```

(gdb) si
0x00007ea9 in ?? ()
1: x/5i $pc
=> 0x7ea9:      mov     $0x8,%eax
    0x7eae:      mov     %eax,%ds
    0x7eb0:      mov     %eax,%es
    0x7eb2:      mov     $0x10,%eax
    0x7eb7:      mov     %eax,%ss
(gdb) set architecture i386
The target architecture is assumed to be i386
(gdb) info registers cs
cs                0x20      32

```

⑨输入 `c` 运行完整程序查看输出

● 实验结果展示：

```

1 Ubuntu 软件包 [a-VirtualBox:~/src/2$ qemu-system-x86_64 -hda hd.img -boot c
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
       Automatically detecting the format is dangerous for raw images, write o
       perations on block 0 will be restricted.
       Specify the 'raw' format explicitly to remove the restrictions.
qemu-system-x86_64: warning: TCG doesn't support requested feature: CPUID.01H:EC
X.vmx [bit 5]

run bootloaderon 1.10.2-1ubuntu1)
enter protect mode

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...

```

----- 实验任务 5 -----

- 任务要求：在进入保护模式后，按照如下要求，编写并执行一个自己定义的 32 位汇编程序，要求简单说一说实现思路，并提供结果截图。 要求：使用两种不同的自定义颜色和一个自定义的起始位置(x,y)，使得 `bootloader` 加载

后, 在显示屏坐标(x,y)处开始输出自己的学号+姓名拼音首字母缩写, 要求相邻字符前景色和背景色必须是相互对调的。

● 思路分析:

1. 进入保护模式后从之前的“enter protect mode”更改为学号+拼音即可, 只需要更改 bootloader.asm 中的 protect_mode_begin 后的内容
2. 注意初始位置需要使用坐标转换—— $\text{offset} = (\text{row} * 80 + \text{col}) * 2$
3. 利用循环左移 4 位实现背景颜色和字体颜色的呼唤

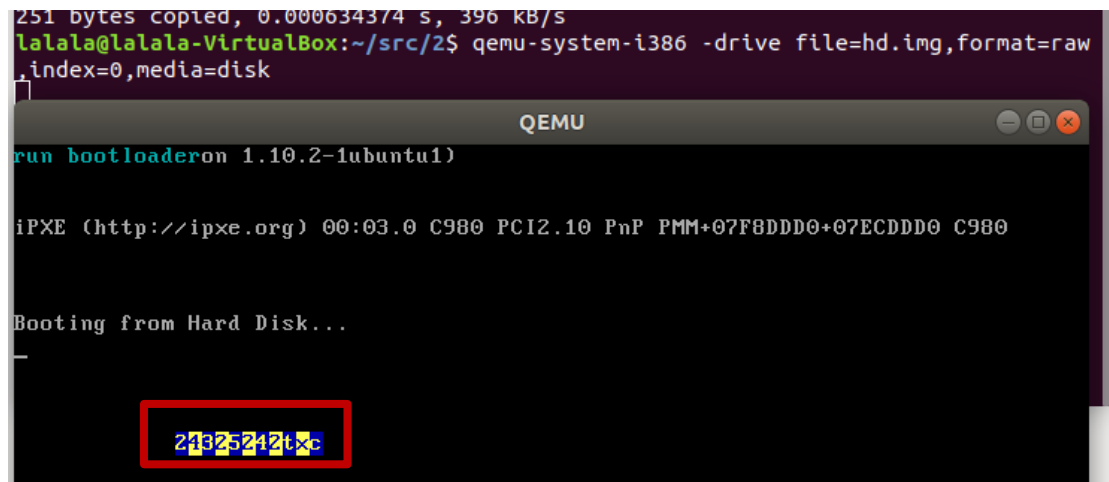
● 实验步骤:

①修改 protect_mode_begin 后内容:

```
1  protect_mode_begin:
2      mov eax, DATA_SELECTOR
3      mov ds, eax
4      mov es, eax
5      mov eax, STACK_SELECTOR
6      mov ss, eax
7      mov eax, VIDEO_SELECTOR
8      mov gs, eax          ; gs 指向显存段
9
10     ; 起始位置 (12, 12)
11     mov ebx, 1944
12
13     mov esi, student_info
14     mov ecx, info_end - student_info
15
16     ; 蓝底黄字
17     mov ah, 0x1E
18
19     .render_loop:
20         mov al, [esi]
21         mov [gs:ebx], ax      ; 将字符(al)和颜色(ah)写入显存
22
23         ; 颜色对调, 利用位循环左移
24         rol ah, 4
25
26         add ebx, 2
27         inc esi
28         loop .render_loop
29
30     jmp $
31
32     student_info db '24325242txc'
33     info_end:
```

②重新编译 Bootloader 并将编译好的二进制写入磁盘镜像，最后使用 qemu 运行

● 实验结果展示：



```
251 bytes copied, 0.000634374 s, 396 KB/s
lala@lala-VirtualBox:~/src/2$ qemu-system-i386 -drive file=hd.img,format=raw,index=0,media=disk
run bootloader on 1.10.2-1ubuntu1)
iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980
Booting from Hard Disk...
24325242tc
```

Section 5 实验总结与心得体会

实验总结：本次实验主要突破了 512 字节限制，实现更大规模程序的运行，掌握了硬件的交互（LBA 模式），还学习了实模式终端（CHS 模式）来读取硬盘，最终实现了从 16 位实模式跳转至 32 位保护模式，并在保护模式输出指定内容

心得体会：

1. 调试器的视角必须与处理器的状态实时同步，CPU 在 16 位模式时必须 set architecture i8086，而在 32 位模式时需要切换 i386
2. 区别并且正确使用远跳转和近跳转
3. 学习巩固了 GDB 调试相关的内容（设置断点、查看寄存器和跳过 loop 等操作）
4. 对进入保护模式的步骤有了更深刻的理解
5. 对寄存器值得保护（不被污染）有了更深得理解，否则很可能导致结果不能输出

Section 6 代码清单

1.bootloader2.asm

2.mbr1.asm(CHS 模式)

3.mbr2.asm(LBA 模式)